

Programming Assignment 1: Build a Game with raylib

Computer Programming (010153002) | Semester 1/2026

Posted: Thursday, 17 September 2026 | **Team size:** exactly 3 students per group

Submission: Code + live presentation, on a day **TBA** during the week following the Final Examination period

The Mission

Your team's mission is to design and build a **playable game** using **raylib**, a simple C library for graphics, input, and audio.

Your goal: a complete, working game — written in **plain C**, using only the programming concepts covered in this course — that your team can confidently explain, line by line, in front of the class.

This is not just a coding exercise. It is a test of whether your team truly **understands** every piece of code you submit.

The One Rule That Matters Most

If you (or your teammates) cannot explain a piece of code, or it does not relate to what we've studied in class, it earns you no score for that part — no matter how impressive it looks.

Copy-pasted tutorial code, AI-generated boilerplate no one understands, and unrelated libraries/features are all treated the same way: **unexplainable code = zero credit for that section**, and it may reduce your whole-project score.

Scope: What You May Use

Allowed:

- **Plain C** (the course's C17/K&R style) — no C++ classes, STL, or .cpp files
- **raylib's C API only** — `InitWindow`, `BeginDrawing/EndDrawing`, `DrawText`, `DrawRectangle`, `IsKeyDown`, `LoadSound`, etc. (raylib is your “screen and controller,” not a shortcut around writing your own game logic)
- Every concept from **Lectures 1–10** of this course (see the checklist below)

Not allowed / flagged as “unrelated code”:

- C++ features (classes, `std::vector`, templates, `new/delete`, ...)
- External game engines, physics/AI libraries, or frameworks beyond raylib
- Multithreading, networking, or other techniques never covered in class
- Large chunks of code lifted from tutorials that your team cannot explain

Required Topic Coverage

Your game's **logic** (not just its graphics) must clearly use the concepts below. You will present a **coverage matrix** showing where each one lives in your code. Your team should aim to **clearly demonstrate at least 8 of these 9** in working, explainable code. Using a topic just to “check a box” without a real purpose will not count.

#	Topic (Lecture)	Typical use in a game
1	Data types, variables, constants	Player stats, screen size constants
2	Expressions & operators	Score math, collision checks
3	Selection (if/switch)	Game states, win/lose conditions
4	Repetition (loops)	The main game loop, animations
5	Functions, macros, recursion	Modular update/draw functions
6	Arrays (1D/2D) + sorting/searching	Entities, grids, leaderboard
7	Pointers & dynamic memory (malloc/free)	Growing lists of bullets/enemies
8	Strings	Player name, HUD text, messages
9	struct / union / enum	Game objects, game-state enum

Game Idea Starters

Pick one, combine ideas, or bring your own (check with the instructor first):

- **Snake** — array/grid of segments, struct for each segment, growing the snake with dynamic memory
- **Breakout / Brick Breaker** — 2D array of bricks (struct per brick), collision via expressions
- **Space Shooter** — dynamic array (pointers) of bullets/enemies, struct per entity, sorted high-score list
- **Memory / Card-Matching Game** — 2D array shuffled with `rand()`, struct per card, string comparison
- **Tic-Tac-Toe / Connect Four** (vs. simple AI) — 2D array board, recursion for the AI, enum for game state
- **Maze Game** — 2D array maze, recursion or loops for movement/pathfinding

Getting Started: Minimal raylib Skeleton

```
#include "raylib.h"

int main(void) {
    const int screenWidth = 800;
    const int screenHeight = 450;

    InitWindow(screenWidth, screenHeight, "My Game");
    SetTargetFPS(60);

    while (!WindowShouldClose()) {
        // 1. Update game state (your functions, structs, arrays...)

        // 2. Draw
        BeginDrawing();
            ClearBackground(RAYWHITE);
            DrawText("Hello, raylib!", 190, 200, 20, LIGHTGRAY);
        EndDrawing();
    }

    CloseWindow();
    return 0;
}
```

Install raylib and see build instructions for your OS at [raylib.com](https://www.raylib.com) (Quickstart).

Weekly Progress Checkpoints

Before the final code submission and presentation, each team checks in **three times**, about once a week, so problems get caught early instead of the night before the deadline. Post each checkpoint to

the course portal: a short status note (a few sentences) plus your current source code (or a link/clip showing it running).

Checkpoint	Target date	What to show
1 — Concept & Setup	Thu, 24 Sep 2026	Team roster confirmed; game concept chosen; raylib toolchain working (a window opens and draws something, even just a colored rectangle)
2 — Core Loop	Thu, 1 Oct 2026	The main game loop updates and draws every frame; at least one <code>struct</code> and one array already in use for real game state
3 — Feature-Complete Draft	Thu, 8 Oct 2026	All core mechanics implemented and playable; coverage matrix draft showing at least 5 of the 9 required topics in use

These checkpoints are low-stakes and not separately scored, but a team that skips two or more of them without a good reason should expect this to be reflected in the **Teamwork & Individual Contribution** portion of the final rubric below.

Deliverables

Submit a single ZIP file containing:

1. **Source Code** — organized `.c/.h` files (no single 1000-line `main.c` — use functions!)
2. **Coverage Matrix** — a short document (or slide) mapping each topic from Lectures 1–10 to the exact file/function where it's used
3. **Design Report (PDF)** — brief description of the game, a flowchart of the main game loop, and each team member's contribution
4. **Compile Instructions** — the exact command(s) to build your game (no Makefile knowledge required — a plain `gcc ... -lraylib ...` command is fine)

The 15-Minute Presentation

Each group presents **live, in front of the class**, for **15 minutes total**:

Time	Segment
0–2 min	Game concept & design overview (whole team)
2–8 min	Live gameplay demo
8–13 min	Code walkthrough — each member explains the part they personally wrote
13–15 min	Instructor Q&A — any member may be asked about any part of the code

All 3 members must speak and must each be able to answer questions about the whole project, not only their own section.

Grading Rubric (100 points)

Category	Points	Focus
Core Gameplay & Functionality	40	Game runs, is playable, has clear win/lose logic
Concept Coverage & Correct Use	25	Correct, purposeful use of ≥ 8 topics from the checklist
Code Quality & Organization	15	Functions, meaningful names, no dead-/unrelated code
Presentation & Understanding	10	Every member explains their code clearly and fluently
Teamwork & Individual Contribution	10	Balanced contribution across all 3 members

Reminder: any code a team cannot explain, or that falls outside course topics, scores **0** for that portion — regardless of how the rest of the rubric is scored.

Submission

- **What:** One ZIP per team (source code + coverage matrix + design report PDF)
 - **Where:** Course portal
 - **When:** Code submission and live presentation both happen on the same **TBA** day, sometime in the week following Final Examinations — exact day and venue will be posted on the course portal once scheduled
- Only one submission per team — make sure all 3 names are listed in the design report.

Good luck! Build something you're proud of — and something every teammate can explain with confidence.